FLAGSHIP PORTFOLIO PROJECT

DOCUMENT 1 OF 5

PRD

Infralytix

AI-Powered Infrastructure Intelligence Platform

Vision & Product Requirements Document — defining the product vision, business goals, target users, the ten Intelligence Modules, and the four-phase roadmap that takes Infralytix from zero to a production-grade AI DevOps platform.

React 19 + TypeScript | FastAPI + Python 3.12 | Gemini AI Agents | PostgreSQL | Docker · Multi-Cloud

Prepared for: Sneharsha — Portfolio Engineering

Version 1.0 · July 2026

Table of Contents

Vision & Product Requirements Document

01 Executive Summary

The one-page version

Infralytix is an AI-powered Infrastructure Intelligence Platform that helps developers, DevOps engineers, and cloud teams analyze applications, automate deployments, diagnose infrastructure issues, optimize cloud resources, and generate DevOps configurations — through a single intelligent interface, powered by a team of specialized AI agents rather than one generic chatbot.

Where traditional monitoring tools stop at displaying metrics, Infralytix **understands** infrastructure: it explains failures in plain language, recommends concrete fixes, and can generate the corrected configuration itself. It is positioned as the connective tissue between the ten-plus disconnected tools a modern developer juggles daily — Docker, GitHub Actions, Kubernetes, cloud consoles, Grafana, Prometheus, and Terraform among them.

10	8	4	~11wk
INTELLIGENCE MODULES	SPECIALIZED AI AGENTS	BUILD PHASES	ZERO → HERO TIMELINE

POSITIONING STATEMENT

Infralytix is not another dashboard. It is an AI engineer that sits inside your deployment lifecycle — reading your repo, your logs, your infrastructure, and your cloud bill — and tells you what's wrong, why, and how to fix it, in the same breath it takes to generate the fix.

02 Vision, Mission & Problem Statement

Why Infralytix needs to exist

Vision Statement

To become the **intelligent operating system for modern software infrastructure** — enabling developers to build, deploy, monitor, optimize, and troubleshoot applications through AI-driven automation, instead of stitching together a dozen disconnected consoles by hand.

Mission

Build an AI engineer that assists throughout the entire software deployment lifecycle: from the moment code is written, through containerization and CI/CD, into production monitoring, cost optimization, and incident response.

The Problem

Modern software development requires developers to work across many disconnected tools — Docker Desktop, GitHub Actions, Jenkins, the Kubernetes Dashboard, the AWS Console, Azure Portal, Grafana, Prometheus, Terraform, assorted CI/CD pipelines, and monitoring systems. Developers spend significant time simply switching between these tools to troubleshoot deployment failures, analyze logs, optimize infrastructure, estimate cloud costs, and configure DevOps workflows. This fragmentation increases complexity, slows development, and makes DevOps disproportionately hard for individuals and small teams who don't have a dedicated platform team.

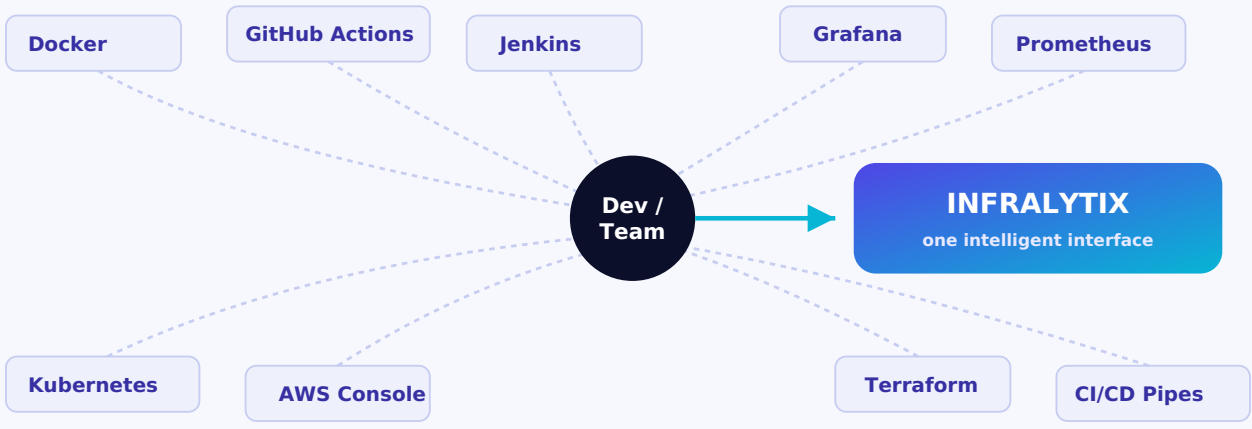

Fig. 1 — Today's fragmented toolchain forces context-switching across 10+ surfaces; Infralytix consolidates the reasoning layer into one interface.

Infralytix addresses this by providing an AI-powered unified platform that analyzes infrastructure, diagnoses issues, generates deployment configurations, and automates repetitive DevOps tasks — replacing "search Stack Overflow and tab-hop between five dashboards" with "ask the platform."

03 Target Users & Personas

Who Infralytix is built for

Segment	Who they are	Why they need Infralytix
Python / Full-Stack Developers Primary	Build applications end-to-end but aren't DevOps specialists	Want deployment and infra handled without becoming a Kubernetes expert
DevOps / Cloud Engineers Primary	Own pipelines, infra, and incident response	Want faster root-cause analysis and less manual log spelunking
Startup Teams & Freelancers Primary	Ship fast with no dedicated platform team	Need an "AI DevOps hire" they can't afford to actually recruit
Students Secondary	Learning cloud & DevOps concepts	Want explanations, not just error codes
SaaS Startups & Engineering Teams Secondary	Scaling teams with growing infra complexity	Need cost visibility and security guardrails as they grow

Representative Persona — "Priya, Indie Full-Stack Developer"

GOALS

Ship a side project to production without learning Kubernetes YAML from scratch; understand why a deploy failed in under two minutes; keep AWS bill predictable.

FRUSTRATIONS

Bounces between Docker Desktop, GitHub Actions logs, and the AWS Console to debug one failed deploy; copy-pastes stack traces into a chat AI with no infra context.

04 Product Goals & Success Metrics

What "good" looks like, quantified

Goal	Metric	Target (v1.0)
Reduce time-to-diagnosis for failures	Median time from log paste to root-cause explanation	< 30 seconds
Reduce tool-switching	% of DevOps tasks completable inside Infralytix without leaving the app	> 70% of Phase 1–2 scope
Deployment config accuracy	% of generated Dockerfiles / GitHub Actions that build without manual edits	> 85%
Cost transparency	Estimate error vs. actual AWS/Azure/GCP bill	within ±15%
Security coverage	Known exposed-secret patterns detected in scanned repos	> 90% of OWASP-listed patterns
Portfolio impact	Demonstrates layered architecture + multi-agent AI design end-to-end	Deployed, documented, demo-ready

DEFINITION OF "HERO" (V1.0 DONE)

All four roadmap phases shipped; ten Intelligence Modules functional; multi-agent Orchestrator routing live; deployed publicly with a recorded walkthrough — a project that reads as a real product, not a bootcamp exercise.

05 The Ten Intelligence Modules

Called "Intelligence Modules," not "features" — each maps to a specialized AI agent

1. Project Intelligence

Analyzes uploaded repositories: architecture, tech stack, dependencies, folder structure, and improvement suggestions.

2. Deployment Intelligence

Generates Dockerfiles, Docker Compose files, GitHub Actions workflows, environment templates, and deployment guides.

3. Infrastructure Intelligence

Monitors CPU, memory, disk, containers, and running services in real time, with AI-generated insights.

4. Log Intelligence

Reads Python tracebacks, Docker/server/Nginx/system logs, and explains root cause, severity, and solution.

5. Cloud Intelligence

Calculates AWS / Azure / GCP estimated costs and surfaces concrete optimization suggestions.

6. Security Intelligence

Detects exposed secrets, weak configurations, Docker risks, environment issues, and vulnerabilities.

7. Repository Intelligence

Scans GitHub repos for dead code, duplicate code, oversized files, missing docs, and architecture issues.

8. DevOps Intelligence

Creates CI/CD pipelines, deployment scripts, infrastructure documentation, and monitoring setup.

9. Architecture Intelligence

Automatically generates system architecture, component diagrams, dependency graphs, and deployment diagrams.

10. AI Engineer

Acts like a senior DevOps engineer: "Why is deployment failing?", "Optimize my Dockerfile," "Generate Terraform."

PRODUCT PRINCIPLE

Most AI tools answer questions. Infralytix analyzes, reasons, recommends, and generates. It doesn't stop at "your Dockerfile has too many layers" — it explains why, proposes the fix, and can generate the corrected file. Same principle for logs, pipelines, cloud costs, and repo analysis.

06 Competitive Landscape

Where Infralytix sits relative to existing tools

Category	Examples	What they do / don't do
Monitoring dashboards	Grafana, Datadog	Show metrics and alerts, but don't explain root cause or generate fixes
CI/CD platforms	GitHub Actions, Jenkins	Execute pipelines but require humans to author and debug them
Cloud cost tools	AWS Cost Explorer	Single-cloud only; no cross-cloud comparison or AI reasoning
General AI chat assistants	ChatGPT, generic copilots	Can explain a pasted error, but have zero live context on your actual infra, repo, or costs
Infralytix	—	**Unifies analysis + reasoning + generation across repo, deploy, infra, cost, and security — with live context via specialized agents**

07 Multi-Agent Product Architecture

Why Infralytix is a team of agents, not one chatbot

To make Infralytix genuinely stand out, it is not built around a single AI chatbot. Instead, it is built around eight specialized AI agents, each responsible for one Intelligence Module's domain, coordinated by a central **Orchestrator** that routes a request to the right agent (or agents) and combines the results into one unified report.

Agent	Owns
Project Intelligence Agent	Repo architecture, stack detection, dependency graph
Deployment Intelligence Agent	Dockerfiles, Compose files, CI/CD generation
Log Intelligence Agent	Log parsing, root-cause explanation
☁ Cloud Intelligence Agent	Cost estimation across AWS/Azure/GCP
🛡 Security Intelligence Agent	Secret scanning, config hardening
Infrastructure Intelligence Agent	Live system & container monitoring
🏗 Architecture Intelligence Agent	Diagram generation
Report Intelligence Agent	Synthesizes multi-agent output into one report

Full orchestration flow, sequence diagrams, and API contracts are detailed in the Software Design Document (Document 3 of 5).

08 Development Roadmap

Four milestones, ~11 weeks, zero to hero

1 Phase 1 — Foundation 2–3 weeks

Get the skeleton and first Intelligence Modules alive

- User authentication
- Dashboard
- Project upload
- Project Intelligence
- Log Intelligence

2 Phase 2 — DevOps Automation 2 weeks

Generate, not just observe

- Docker reviewer
- Deployment generator
- GitHub repository scanner
- Report generation

3 Phase 3 — Infrastructure Intelligence 2–3 weeks

Live signal + cost + security

- Live system monitoring
- Cloud cost estimator
- Security analysis
- Architecture diagrams

4 Phase 4 — Advanced AI Platform 3–4 weeks

The "hero" layer

- Multi-agent AI architecture (Orchestrator live)
- Kubernetes support
- Terraform generation
- RAG-based documentation assistant
- Team collaboration
- Advanced analytics

SEQUENCING RATIONALE

Phases 1–2 prove the core loop (analyze → explain → generate) fast, so there's a demoable product within three to five weeks. Phases 3–4 layer on the harder, more differentiated capabilities — live monitoring, cost intelligence, and the true multi-agent Orchestrator — once the foundation is stable.

09 Risks, Assumptions & Open Questions

Going in with eyes open

Risk	Impact	Mitigation
Gemini API cost/latency at scale	Slower diagnosis, higher run cost	Cache repeated analyses; stream partial responses; local Ollama fallback (Phase 4, future)
Multi-agent orchestration complexity	Harder to debug than a single model call	Ship single-agent flows first (Phase 1–2), introduce Orchestrator in Phase 4 once individual agents are proven
False positives in Security Intelligence	Alert fatigue, lost trust	Start with high-confidence, well-known patterns; expand coverage iteratively
Cloud cost estimates diverge from real bills	Undermines Cloud Intelligence credibility	Be explicit about assumptions; show ranges, not false-precision single numbers
Scope creep across 10 modules	Nothing ships	Hold the phase boundaries firmly; Phase 1–2 must be demoable before Phase 3 starts

Assumptions

- Single-developer build (portfolio project) — architecture favors clarity and modularity over premature scale.
- Gemini API is the default AI provider; LangChain and local Ollama models are explicitly deferred ("future") per the technology stack.
- Kubernetes and Terraform support are Phase 4 stretch goals, not Phase 1 requirements.

10 Appendix — Technology Stack

Quick reference; full rationale in the SDD

Layer	Technologies
Frontend	React 19, TypeScript, TailwindCSS, React Router, Axios, Chart.js, React Flow (architecture diagrams), Monaco Editor (code viewing/editing)
Backend	Python 3.12+, FastAPI, SQLAlchemy, Alembic, Pydantic, Uvicorn **Celery — future** **Redis — future**
AI Layer	Gemini API **LangChain — optional** **RAG — future** **Ollama — future**
Database	PostgreSQL (production), SQLite (development)
DevOps	Docker, Docker Compose, GitHub Actions
Monitoring	psutil, Docker SDK **Prometheus — future** **Grafana — future**
Cloud	AWS, Azure, GCP (cost + optimization estimation across all three)
Security	JWT Authentication, OAuth (GitHub Login), bcrypt, HTTPS, Role-Based Access Control (RBAC)

NEXT DOCUMENT IN THIS SUITE

Document 2 — Software Requirements Specification (SRS): functional & non-functional requirements written to IEEE 830, use cases, and interface requirements for every module above.